Merkle Tree LMDB Backend Refactor Plan

Plan: Trait-Based Merkle Node Storage Abstraction

Context

The merkle tree is stored on disk using a custom binary file format (`MerkleNodeDB`). Each node lives in `.oxen/tree/nodes/{prefix}/{suffix}/` with two files: `node` (header + child index) and `children` (msgpack-serialized child data). This format was chosen over RocksDB for faster open times, but we want to explore LMDB as an alternative backend that could consolidate thousands of small files into a single memory-mapped database.

A single repository uses one format — no inter-mixing. The code should support either format at a single commit, with a migration script for clean cutover. Later, the file-based format can be dropped.

The goal is to: 1. **Phase 1**: Introduce a trait abstraction over merkle node storage and refactor the existing file-based code to implement it 2. **Phase 2**: Add an LMDB-based backend behind that trait

---

Key Design Decisions

`TMerkleTreeNode` is now object-safe (DONE in PR #406)

`TMerkleTreeNode` no longer requires `Serialize` — it has a `to_msgpack_bytes() -> Result<Vec<u8>, rmp_serde::encode::Error>` method with a blanket impl for any `T: Serialize + MerkleTreeNodeIdType + Debug + Display`. This makes all storage traits fully object-safe — no enum dispatch needed.

Bonus changes from the same PR that simplify downstream work: - `MerkleTreeNodeType::from_u8()` now returns `Result<_, InvalidMerkleTreeNodeType>` instead of panicking - `MerkleNodeDB::to_node()` returns the more specific `rmp_serde::decode::Error` instead of `OxenError` - New `OxenError::MerkleTreeError` variant wraps `InvalidMerkleTreeNodeType`

Separate read and write interfaces

Read and write paths have zero overlap. Callers never mix them. Keeping them separate makes intent clear and avoids unnecessary capabilities.

Backend injected via `LocalRepository`

Add a `merkle_store()` method on `LocalRepository` that returns `MerkleNodeStore`. The store is constructed lazily based on which backend is detected on disk. This minimizes signature changes across the 9 caller files.

`to_node()` deserialization moves to model layer

`MerkleNodeDB::to_node(dtype, data)` is format-independent msgpack deserialization. It becomes `EMerkleTreeNode::from_type_and_bytes()` in the model layer, shared by both backends.

Caching stays external to the trait

The existing LRU cache in `merkle_tree_node_cache.rs` wraps `MerkleTreeNode::from_hash()` / `read_children_from_hash()`. This remains unchanged — it sits above the storage trait.

`dir_hashes` stays separate

The dir_hashes RocksDB (path -> hash mapping per commit) is a different abstraction. Not unified in this change.

Phase 1: Introduce Trait + File Backend

Step 1.0: ✅ DONE — Make `TMerkleTreeNode` object-safe (PR #406)

Merged in commit `086d3243c`. Summary of what landed:

- `TMerkleTreeNode` no longer requires `Serialize`; it now has `fn to_msgpack_bytes(&self) -> Result<Vec<u8>, rmp_serde::encode::Error>`
- Blanket impl `impl<T: Serialize + MerkleTreeNodeIdType + Debug + Display> TMerkleTreeNode for T` covers all 5 node types (the manual `impl TMerkleTreeNode for X {}` blocks were removed)
- Call sites in `MerkleNodeDB::write_node` and `MerkleNodeDB::add_child` switched from `.serialize(&mut Serializer::new(&mut buf))` to `.to_msgpack_bytes()?`
- Generic bounds simplified to `N: TMerkleTreeNode` (no longer needs `+ Serialize`)
- Bonus: `MerkleTreeNodeType::from_u8()` returns `Result<_, InvalidMerkleTreeNodeType>` instead of panicking; `MerkleNodeDB::to_node()` returns `Result<EMerkleTreeNode, rmp_serde::decode::Error>`; new `OxenError::MerkleTreeError` variant

Step 1.1: Add `EMerkleTreeNode::from_type_and_bytes()`

Extract `MerkleNodeDB::to_node()` (which already returns the specific serde error after PR #406) into a method on `EMerkleTreeNode`:

```rust
// crates/lib/src/model/merkle_tree/node.rs
impl EMerkleTreeNode {
    pub fn from_type_and_bytes(
        dtype: MerkleTreeNodeType,
        data: &[u8],
    ) -> Result<Self, rmp_serde::decode::Error> {
        // exact same body as current MerkleNodeDB::to_node()
    }
}
```

Update `MerkleNodeDB::to_node()` to delegate to this. No caller changes yet.

Files: `crates/lib/src/model/merkle_tree/node.rs`, `crates/lib/src/core/db/merkle_node/merkle_node_db.rs`

Step 1.2: Define the storage abstractions

Create `crates/lib/src/core/db/merkle_node/store.rs`:

```rust
use crate::error::OxenError;
use crate::model::{MerkleHash, MerkleTreeNodeType, TMerkleTreeNode};
use crate::model::merkle_tree::node::{EMerkleTreeNode, MerkleTreeNode};

/// Metadata returned when reading a single node.
pub struct MerkleNodeRecord {
    pub hash: MerkleHash,
    pub dtype: MerkleTreeNodeType,
    pub parent_id: Option<MerkleHash>,
    pub node: EMerkleTreeNode,
    pub num_children: u64,
}

/// Read-only access to merkle node storage (object-safe).
pub trait MerkleNodeReader: Send + Sync {
    fn exists(&self, hash: &MerkleHash) -> bool;
    fn get_node(&self, hash: &MerkleHash) -> Result<Option<MerkleNodeRecord>, OxenError>;
    fn get_children(&self, hash: &MerkleHash) -> Result<Vec<(MerkleHash, MerkleTreeNode)>,
        OxenError>;
}

/// Write session for building a node (object-safe thanks to Step 1.0).
pub trait MerkleNodeWriter: Send + Sync {
    fn create_node(
        &self,
        node: &dyn TMerkleTreeNode,
        parent_id: Option<MerkleHash>,
    ) -> Result<Box<dyn MerkleNodeWriteSession>, OxenError>;

    fn create_node_if_absent(
        &self,
        node: &dyn TMerkleTreeNode,
        parent_id: Option<MerkleHash>,
    ) -> Result<Option<Box<dyn MerkleNodeWriteSession>>, OxenError>;
}

/// A write session for a single node being constructed.
pub trait MerkleNodeWriteSession {
    fn node_id(&self) -> MerkleHash;
    fn add_child(&mut self, child: &dyn TMerkleTreeNode) -> Result<(), OxenError>;
    fn finish(self: Box<Self>) -> Result<(), OxenError>;
}

/// Top-level store combining read + write access for a backend.
pub trait MerkleNodeStore: MerkleNodeReader + MerkleNodeWriter {}
impl<T: MerkleNodeReader + MerkleNodeWriter> MerkleNodeStore for T {}
```

Files: NEW `crates/lib/src/core/db/merkle_node/store.rs`

Step 1.3: Create the file backend

Rename `merkle_node_db.rs` to `file_backend.rs`. Refactor into:

- `FileBackend` struct (holds `tree_nodes_dir: PathBuf`) — implements `MerkleNodeReader` + `MerkleNodeWriter`
- `FileWriteSession` struct (the current write-mode `MerkleNodeDB` fields: node_file, children_file, data_offset, node_id, etc.) — implements `MerkleNodeWriteSession`
- Keep `MerkleNodeLookup`, `node_db_prefix()`, `NODE_FILE`, `CHILDREN_FILE` as private implementation details

The `MerkleNodeReader` impl for `FileBackend`: - `exists()`: check `node_db_path.join("node").exists() && .join("children").exists()` - `get_node()`: open read-only, load lookup, return `MerkleNodeRecord` - `get_children()`: open read-only, call `map()`, return vec

The `MerkleNodeWriter` impl for `FileBackend`: - `create_node()`: create dirs, open files, write header via `to_msgpack_bytes()`, return `Box<FileWriteSession>` - `create_node_if_absent()`: check `exists()` first, delegate to `create_node()` if absent

`FileWriteSession` impl of `MerkleNodeWriteSession`: - `node_id()`: return stored hash - `add_child()`: serialize via `child.to_msgpack_bytes()`, write to node+children files - `finish()`: flush and sync both files

Files: `crates/lib/src/core/db/merkle_node/merkle_node_db.rs` -> `crates/lib/src/core/db/merkle_node/file_backend.rs`

Step 1.4: Update module structure

Update `crates/lib/src/core/db/merkle_node.rs`:

```rust
pub mod store;
pub mod file_backend;

pub use store::{MerkleNodeStore, MerkleNodeReader, MerkleNodeWriter, MerkleNodeWriteSession,
        MerkleNodeRecord};
pub use file_backend::FileBackend;
```

Files: `crates/lib/src/core/db/merkle_node.rs`

Step 1.5: Add `merkle_store()` to `LocalRepository`

Add a method that constructs the store. A single repo uses one format, detected by what's on disk:

```rust
impl LocalRepository {
    pub fn merkle_store(&self) -> Box<dyn MerkleNodeStore> {
        // Phase 2 will add LMDB detection here
        Box::new(FileBackend::new(
            self.path
                .join(constants::OXEN_HIDDEN_DIR)
                .join(constants::TREE_DIR)
                .join(constants::NODES_DIR),
        ))
    }
}
```

Files: `crates/lib/src/model/repository/local_repository.rs`

Step 1.6: Migrate read-path callers

1.6a: `MerkleTreeNode::from_hash()` and `read_children_from_hash()` (`crates/lib/src/model/merkle_tree/node/merkle_tree_node.rs`)

These are the two core read functions. Currently:

```rust
fn from_hash_uncached(repo: &LocalRepository, hash: &MerkleHash) -> Result<Self, OxenError> {
    let node_db = MerkleNodeDB::open_read_only(repo, hash)?;
    ...
}
```

Change to:

```rust
fn from_hash_uncached(repo: &LocalRepository, hash: &MerkleHash) -> Result<Self, OxenError> {
    let store = repo.merkle_store();
    let record = store.get_node(hash)?
        .ok_or_else(|| OxenError::basic_str(format!("Node not found: {}", hash)))?;
    Ok(MerkleTreeNode {
        hash: *hash,
        node: record.node,
        parent_id: record.parent_id,
        children: Vec::new(),
    })
}
```

Similarly for `read_children_from_hash_uncached` -> `store.get_children(hash)`.

1.6b: `CommitMerkleTree` (`crates/lib/src/core/v_latest/index/commit_merkle_tree.rs`)

Replace all `MerkleNodeDB::exists(repo, hash)` calls with `repo.merkle_store().exists(hash)`. The rest delegates to `MerkleTreeNode::from_hash` which is already migrated.

Step 1.7: Migrate write-path callers

For each file, replace: - `MerkleNodeDB::open_read_write(repo, &node, parent_id)` -> `repo.merkle_store().create_node(&node, parent_id)?` - `MerkleNodeDB::open_read_write_if_not_exists(repo, &node, parent_id)` -> `repo.merkle_store().create_node_if_absent(&node, parent_id)?` - `db.add_child(&child)` -> `session.add_child(&child)` - `db.close()` -> `session.finish()` - `db.node_id` -> `session.node_id()`

Caller files (in order of complexity):

1. `crates/lib/src/core/v_latest/commits.rs` — 2 usages
2. `crates/lib/src/core/v_latest/entries.rs` — 3 usages
3. `crates/lib/src/repositories/tree.rs` — 1 usage
4. `crates/lib/src/repositories/commits/commit_writer.rs` — ~6 usages (most complex, nested sessions)
5. `crates/lib/src/command/migrate/m20250111083535_add_child_counts_to_nodes.rs` — 3 usages

Step 1.8: Handle the v0.19.0 compat layer

`crates/lib/src/core/v_old/v0_19_0/index/commit_merkle_tree.rs` uses `MerkleNodeDB` directly. Since this is legacy code reading the file format specifically, it can continue using `FileBackend` directly rather than going through the trait.

Phase 2: LMDB Backend

Step 2.1: Add `heed` dependency

Add to `crates/lib/Cargo.toml`:

```
[dependencies]
heed = { version = "0.22" }
```

Step 2.2: LMDB schema

Single LMDB environment at `.oxen/tree/lmdb/` with two named databases:

Database	Key (16 bytes)	Value
`nodes`	node hash (u128 LE)	`[u8 type][u128 parent_id][msgpack node data]`
`children`	parent hash (u128 LE)	`[u32 num_children][per child: u8 type + u128 hash + msgpack data length prefix + msgpack data]`

Step 2.3: Implement `LmdbBackend` + `LmdbWriteSession`

Create `crates/lib/src/core/db/merkle_node/lmdb_backend.rs`:

- `LmdbBackend` struct holds `heed::Env` + db handles. Implements `MerkleNodeReader` + `MerkleNodeWriter`.
- `LmdbWriteSession` accumulates children in memory, writes everything in one transaction on `finish()`.
- Environment managed via a global cache (similar to existing `DB_INSTANCES` pattern for RocksDB).

Step 2.4: Update `merkle_store()` for backend detection

```rust
pub fn merkle_store(&self) -> Box<dyn MerkleNodeStore> {
    let lmdb_path = self.path.join(".oxen/tree/lmdb");
    if lmdb_path.join("data.mdb").exists() {
        Box::new(LmdbBackend::open(lmdb_path))
    } else {
        Box::new(FileBackend::new(...))
    }
}
```

Step 2.5: Migration command

Add a CLI subcommand `oxen migrate merkle-to-lmdb` that: 1. Opens file backend as reader 2. Opens LMDB backend as writer 3. Walks every commit's tree, copies nodes

Step 2.6: Tests

Run the existing test suite against both backends. Add an integration test that: 1. Creates a repo, makes commits (file backend) 2. Migrates to LMDB 3. Verifies all reads return identical data

Verification

After Phase 1:

```
cargo fmt --all
cargo clippy --workspace --no-deps -- -D warnings
bin/test-rust
```

All existing tests must pass unchanged since Phase 1 is a pure refactor (file backend only).

After Phase 2: - Run the migration on a test repo - Run the full test suite with `OXEN_MERKLE_BACKEND=lmdb` - Benchmark read performance (particularly `from_hash` and `read_children_from_hash`) comparing file vs LMDB

Critical Files Reference

File	Role
`crates/lib/src/core/db/merkle_node/merkle_node_db.rs`	Current storage impl, becomes `file_backend.rs`
`crates/lib/src/core/db/merkle_node.rs`	Module root, updated for new structure
`crates/lib/src/model/merkle_tree/node.rs`	`EMerkleTreeNode` — gets `from_type_and_bytes()`
`crates/lib/src/model/merkle_tree/node/merkle_tree_node.rs`	Primary read caller (`from_hash`, `read_children_from_hash`)
`crates/lib/src/core/v_latest/index/commit_merkle_tree.rs`	`exists()` checks + tree loading
`crates/lib/src/repositories/commits/commit_writer.rs`	Most complex write caller
`crates/lib/src/repositories/tree.rs`	Write caller for tree modifications
`crates/lib/src/core/v_latest/commits.rs`	Write caller for squash
`crates/lib/src/core/v_latest/entries.rs`	Write caller for entries
`crates/lib/src/command/migrate/m20250111083535_add_child_counts_to_nodes.rs`	Migration write caller
`crates/lib/src/model/repository/local_repository.rs`	`merkle_store()` factory method
`crates/lib/src/model/merkle_tree/node/merkle_tree_node_cache.rs`	Caching layer (unchanged)
NEW `crates/lib/src/core/db/merkle_node/store.rs`	Trait definitions
NEW `crates/lib/src/core/db/merkle_node/lmdb_backend.rs`	Phase 2 LMDB implementation

File	Role
`crates/lib/src/model/merkle_tree/node_type.rs`	`TMerkleTreeNode` (already object-safe — PR #406)